

Analysis: Alien Generator

Test Set 1

We can check for every i , ($1 \leq i \leq G$) whether there exists a k such that

$$\sum_{j=0}^k (i+j) = \sum_{j=0}^k i + \sum_{j=0}^k j = ((k+1) \times i) + \frac{k \times (k+1)}{2} = G$$

Finding such a k (if one exists) can be done by [binary searching](#) the range $[0, G]$, and hence takes $O(\log G)$ time. For a candidate k in that range, we check if $((k+1) \times i) + \frac{k \times (k+1)}{2} = G$ and alter the range based on the equality. Time complexity here is $O(G \log G)$.

Alternative solution

For this Test Set, we can implement a brute force solution. We iterate over every i , ($1 \leq i \leq G$) and try to sum up numbers $[i, i+1, i+2, \dots]$ until the sum exceeds or equals G . If the sum equals G , then we increment our result by one. Here, each iteration takes $O(G/i)$ time.

$$\sum_{i=1}^G (1/i) = O(\log(G))$$

Therefore, the overall time complexity of this solution is $O(G \log G)$.

Test Set 2

Since the upper bound on G is 10^{12} , $O(G \log G)$ solution times out. Let us define $H = \lceil \sqrt{2 \times G} \rceil$. An observation can be made that $k \leq H$. Therefore, for each k in the range $[0, H]$, we can binary search for i in the range $[1, G]$ thereby making the total runtime $O(\sqrt{G} \times \log(G))$.

This solution might not pass within the time limit for slow languages. Therefore, we will look at a better solution next.

We can rewrite the equation we saw in Test Set 1 as $i = \frac{2 \times G - k^2 - k}{2 \times (k+1)}$. Next, for each k in the range $[0, H]$, we can check in $O(1)$ whether we can obtain a positive integer value for i that satisfies the above equation. The runtime here is $O(\sqrt{G})$.

Alternative solution

We can dig deeper into the relationship between G , K , and d , the number of days it takes for the machine to produce exactly G gold starting at K on day one. They form the equation

$$\frac{d(K + (K + d - 1))}{2} = G$$

which is equivalent to $d(d + (2K - 1)) = 2G$. Since one of d and $(d + (2K - 1))$ is even and the other is odd, any pair of positive integers x and y such that exactly one of them is even and

$x \times y = 2\mathbf{G}$ can be mapped to them with the smaller of the two being d and the larger one $(d + (2K - 1))$, which is always greater than d . Since each mapping produces a different d , each pair corresponds to a unique solution for d and K . Conversely, every pair of d and K that satisfies the equation corresponds to a different x, y pair.

To count the number of such pairs, let g be the largest odd factor of $2\mathbf{G}$. Note that any (ordered) pair x', y' such that $x' \times y' = g$ corresponds to a pair $x = \frac{2\mathbf{G}}{g}x'$ and $y = y'$. Finally, assume the prime factorization of g is

$$g = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_n^{\alpha_n}$$

the number of such ordered pairs is $(\alpha_1 + 1)(\alpha_2 + 1) \cdots (\alpha_n + 1)$. We can thus prime factorize $\mathbf{G}$, ignore the 2, and multiply all other prime powers accordingly. [Prime factorization](#) can be trivially implemented in $O(\sqrt{\mathbf{G}})$ complexity and there are $o(\log(\mathbf{G}))$ [prime factors](#). Therefore the total time complexity is $O(\sqrt{\mathbf{G}})$.

Analysis: Binary Operator

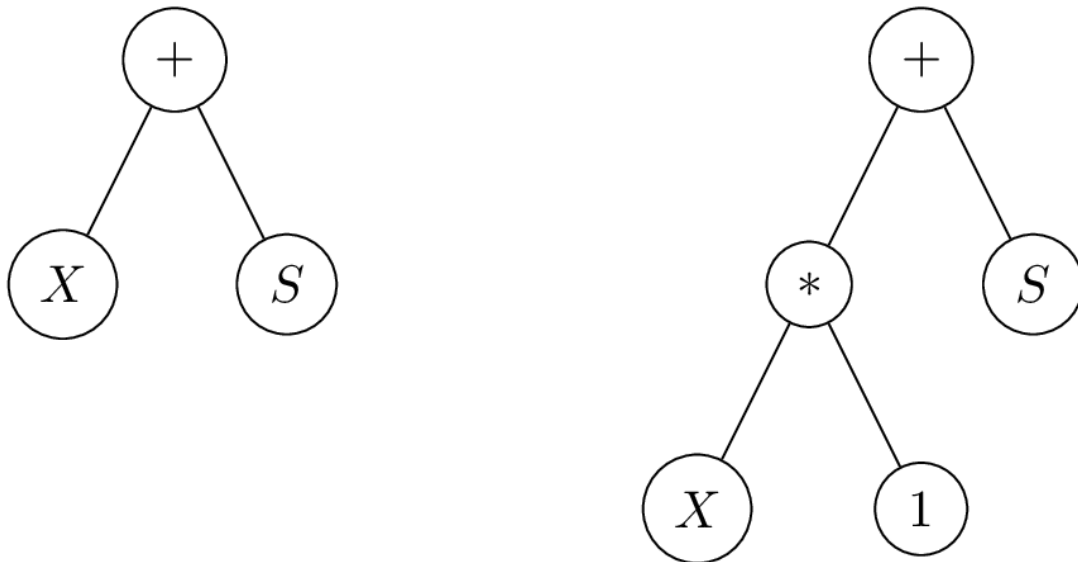
For each expression, we could [construct an expression tree](#) for it and consider it as an [arithmetic tree](#), with, $+$, $*$, $\#$ as its operators. We can then start trying to reduce the whole tree so that we could compare different expressions more easily.

Test Set 1

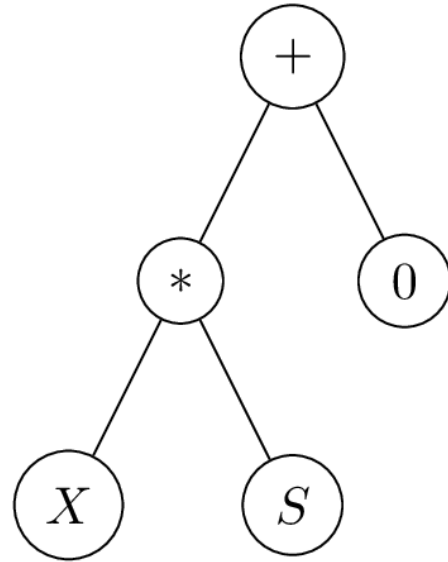
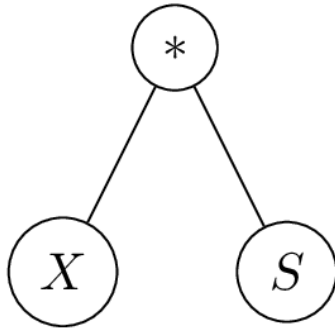
In the first test set, for each expression we could first examine if there is a $\#$ in it. If not, a simple traversal through the arithmetic tree would give us the value represented by the equation. Otherwise, we could start from the node where the operator is $\#$. We can reduce this node's left and right subtrees into a single node each by traversing each subtree and calculating its value accordingly. This will result in a subtree with leaf values L and R , and root $\#$ (i.e. $L\#R$ in expression form). Now we have a tree where there are only leaf nodes under the single $\#$, the next step will be trying to process node $\#$'s ancestors, and ultimately we want to reduce the whole expression into a general form of $(A\#B) * C + D$ in order to do further categorization. For simplicity we would denote $L\#R$ as X from now on.

We would start try to transform the expression into our desired form from the subtree where X is the left leaf. Specifically we want to Consider X 's sibling tree, since there is no $\#$ in that tree, we could use the same traversal method to get its value S . Now consider the parent node of X :

1. X 's parent is a $+$, or X is the root node: we would reconstruct X into a subtree, where its root node is a $*$, and its left node being X , and right node being 1.

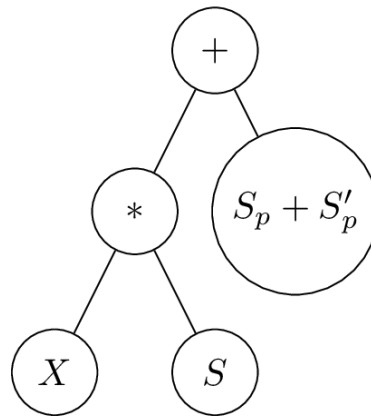
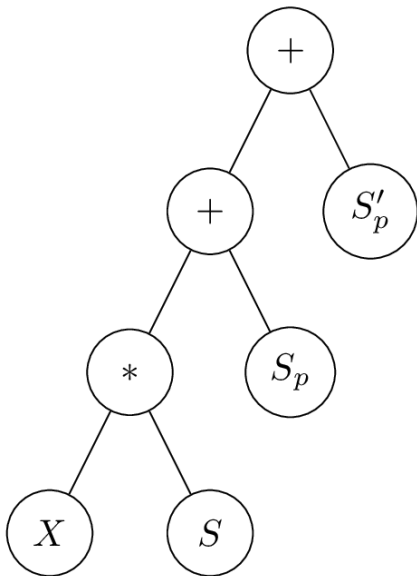


2. X 's parent is a $*$: in this case, we added a $+$ node as a new parent, and the value 0 as the new right-sibling S_p .

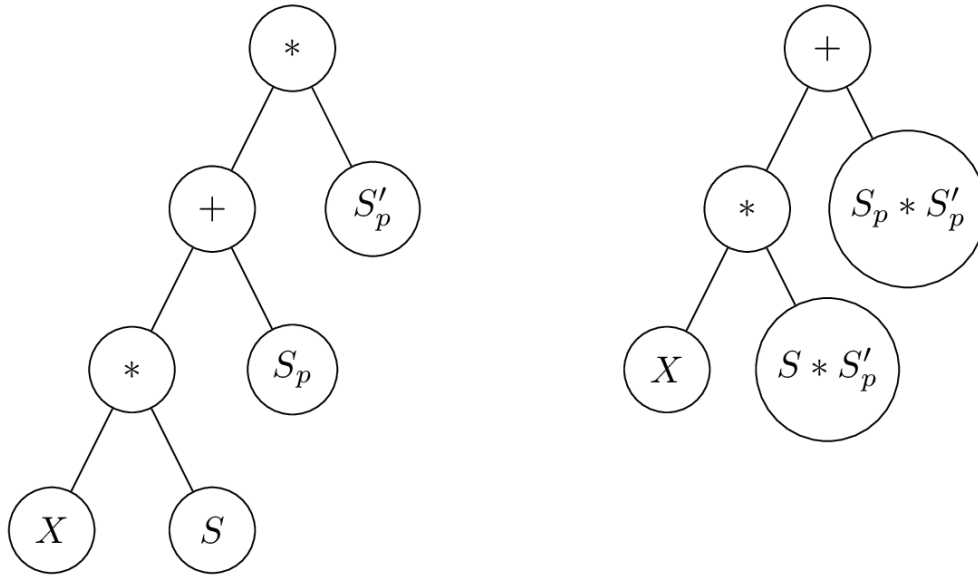


We then consider X 's great-grandparent, and the corresponded reduced sibling S'_p :

1. the node is a $+$, or there is no such node: we could reduce the height of the tree by replacing the tree with its left subtree, and substitute the value of its right subtree from S_p to $S_p + S'_p$



2. the node is a $*$: samely, we would reduce the subtree with its left subtree, but this time we substitute the value of its right subtree with $S_p * S'_p$, and also replace X 's sibling with $S * S'_p$



Notice that, with this process, we could reduce the height of the tree by 1 each time, we could then perform this reduction process recursively until the expression is irreducible.

After the reduction process, the expression would be simplified into the form of $(A \# B) * C + D$, where A, B, C, D are all integers. Therefore the expression can be represented by a 4-tuple (A, B, C, D) . We can then use a hashtable or any similar data structure to categorize them into different equivalence classes. Note that when $C = 0$, the only comparator matters here will be D only, since any value of $A \# B$ does not affect the value at all.

Test Set 2

For the second test set, we could follow the similar solution as mentioned above: reduce the expressions, then categorized into different classes. However, this method would become much more complicated since there might be more than one $\#$ in each expression. As a result, we cannot represent each expression as a 4-tuple, and the reduction rules are much more complicated. In fact, consider the following expression (not fully parenthesized for the sake of simplicity):

$$((0 \# 1) + 1) * ((0 \# 2) + 1) * ((0 \# 3) + 1) * \dots$$

If we utilize the same denotation in test set 1, naming each expression as $x_1, x_2, \dots, x_n$, this will result in a polynomial with n variables, and therefore 2^n coefficients to consider in terms of categorization. Luckily, since $((0 \# 1) + 1)$ already takes 9 characters per block, and there is also need to spend additional characters on enclosing brackets and operators, the total number of coefficients for any given input string will be small enough to allow such deterministic solution to pass.

Alternative solution

There is also a simpler nondeterministic approach to this problem. Since the operator $\#$ represents a total function, for each calculation of some $X \# Y$, we could randomly assign a value as the result, and memorize it for further calculation of the same $X \# Y$. We could utilize it to calculate the exact value of an expression, then state that two expressions belong to the same equivalence class if their corresponding values are the same. This process will never put two expressions from the same class into different classes, but may result in a collision that mistakenly puts two expressions from different classes into the same class. Unfortunately, we do not have a simple proof that such solution has sufficiently small probability of failure. To further decrease the probability of false matches, the process can be repeated several times, putting two expressions into the same class only if their evaluations matched every time.

Analysis: Rock Paper Scissors

Test Set 1

We could observe that it is easy to calculate the expected value of a certain strategy by iterating through all the choices you make every day and adjust your friend's probabilities of picking different choices accordingly. Therefore, we could start from some naive strategy to maximize our chance of winning, one strategy would be something like `RSPRSPRSP...`, and calculate its expected value. If such strategies cannot reach our expected bonus value, we could try to create a random strategy instead, calculate its expected value of bonus, and do so repeatedly until we find a strategy of which the expected bonus is at least $\mathbf{X}$. This approach is enough to solve the first test set.

Test Set 2

For the second test set, since the maximum value of $\mathbf{X}$ is raised, the method for Test Set 1 will exceed the time limit. Therefore we should find a more efficient way to look for a better strategy other than randomly generating them. We could first utilize what we have tried in the first test set, generating some random strategies, and pick the one with the best expected bonus as our base strategy S . We then randomly pick a day in S , and try to change the decision of that day and get a new strategy S' . If the expected bonus of the new strategy $E(S')$ is higher than that of the original $E(S)$, we could substitute S with S' as our best strategy. By keep performing this substitution process, it is obvious the expected value of our strategy will be either increasing or reach a local maximum. Thus we could utilize this method, also known as [hill climbing](#), to find a strategy which has an expected value larger than $\mathbf{X}$. Note that since what we find here is a local maximum, it is possible that we would eventually find out that the best strategy derived from S cannot reach the value of $\mathbf{X}$. In that case we have to reselect the initial S and perform another substitution process.

Test Set 3

For the last test set, the methods we applied in the previous test sets could not find us a strategy in the given time limit. However we could try to solve the problem in a different angle. In this problem, we are targeting an expected bonus of $\mathbf{X}$. To simplify the problem, instead of finding a strategy to reach the targeted bonus, we can try to maximize the expected value we could get by using the best possible strategy we could have. This value will always be at least as large as $\mathbf{X}$ because the solution is guaranteed to exist in the problem statement. We could then use dynamic programming to solve this problem.

Let $v[r][p][s]$ denote the maximum expected value you could get if you made exactly r rocks, p papers, and s scissors in your strategy on the N -th day. We could first observe $r + p + s = N$, and that this value will be affected by the last decision you made in your strategy. For example, if you decide to choose rock on the last day, the expected value will be

$$v[r-1][p][s] + \frac{p}{n-1} \times \mathbf{W} + \frac{s}{n-1} \times \mathbf{E}.$$

Similarly, this can be applied to choosing paper or scissors as the last decision. Thus we could conclude that

$$\begin{aligned}
v[r][p][s] = & \max(v[r-1][p][s] + \frac{p}{n-1} \times \mathbf{W} + \frac{s}{n-1} \times \mathbf{E}, \\
& v[r][p-1][s] + \frac{s}{n-1} \times \mathbf{W} + \frac{r}{n-1} \times \mathbf{E}, \\
& v[r][p][s-1] + \frac{r}{n-1} \times \mathbf{W} + \frac{p}{n-1} \times \mathbf{E})
\end{aligned}$$

We then get the function for state transformation. As for the starting condition, since on the first day our friend's decision is totally random, we could easily have

$$v[1][0][0] = v[0][1][0] = v[0][0][1] = \frac{1}{3}\mathbf{W} + \frac{1}{3}\mathbf{E}$$

Finally, to get the whole strategy to achieve the maximum bonus, we could record the last decision we made with another array $step[i][j][k]$, and backtrace it when we found the final value at $\max(v[i][j][k])$ where $i + j + k = 60$.

We could do some brief analysis on the time and space complexity here. Notice that since in our dynamic programming process, if we want to find the optimal strategy for the N -th day, we will need to calculate all the states where $i + j + k \leq N$, which means that the space we are using will be $O(\binom{N}{3}) = O(N^3)$. As for the time complexity, since our transformation is constant, the dynamic programming process will be $O(\binom{N}{3}) = O(N^3)$. In our problem, where $N = 60$, such complexity is sufficient for our solution to find out the strategy in the given time limit.

Analysis: Smaller Strings

Test Set 1

Let us calculate the upper bound of a brute-force solution in the worst case scenario.

Given the limits of Test Set 1, $1 \leq K \leq 5$ and $1 \leq N \leq 8$, we can compute the total number of strings, in the worst case scenario, to be $K^N = 5^8 = 390625$. This number is low enough to generate all of them, keeping track of the ones that are palindromes and lexicographically smaller than the input string S .

Test Set 2

For Test Set 2, the limits are much higher, so the strategy mentioned above is too slow.

Let us start by making an observation about the definition of lexicographically smaller strings. Say we have two strings A and B of the same length, and i is the first position at which A_i and B_i differ. By definition, if $A_i < B_i$, then A is lexicographically smaller than B . Now notice no matter what characters we append to the end of the strings A and B , we still have that A is lexicographically smaller than B .

Let us define $H = \lceil \frac{N}{2} \rceil$, which is the size of the first half of the input string S , rounded up. Also, let us define S' to be the prefix of length H from the input string S .

To come up with an efficient solution, we only need to count the number of strings T' of size H (not necessarily palindromes) are lexicographically smaller than S' (with the caveat below). The reason is based on the above observation.

Insight 1: If T' is lexicographically smaller than S' , then appending appropriate $N - H$ characters to make it a palindrome of size N , will keep it lexicographically smaller than S .

We can construct a palindrome string T of size N as follows. Let R' be the reverse of T' . If N is even, then combine T' and R' . For example, if $N = 6$ and $T' = "abc"$, then $T = "abccba"$. If N is odd, then combine T' (without the last character) and R' . And for example, if $N = 7$ and $T' = "abcd"$, then $T = "abcdcba"$.

Let us see how these ideas fit together with an example: Let us say that $S = "tomato"$ and $N = 6$. Then, we have that $H = \lceil \frac{6}{2} \rceil = 3$ and $S' = "tom"$. If we choose an arbitrary string $T' = "pea"$, we can observe that $"pea"$ is lexicographically smaller than $"tom"$. Therefore, we can build a palindrome string $T = "peaaep"$, which is guaranteed to be lexicographically smaller than $"tomato"$.

Hence, the problem can now be simplified to finding how many strings of size H (not necessarily palindromes) are lexicographically smaller than S' .

The only **caveat** is for the string $T' = S'$. By just analysing T' , we can not say whether T is going to be lexicographically smaller than S or not. Therefore, for this particular T' we should generate T manually and check if it is lexicographically smaller than S , and if so, add that to our answer.

Also notice we do not have to generate all of the possible lexicographically smaller strings and count them. Notice, if we assume first K letters of the alphabet as digits in K -based numeric

system, i.e. $a = 0, b = 1, \dots, z = 25$, then every string T' lexicographically smaller than S' can be mapped to a number that is smaller than the number corresponding to S' (and vice versa).

Insight 2: In order to calculate the number of strings lexicographically smaller than S' , we just need to convert the $\mathbf{K}$ -based number corresponding S' to base 10.

For example, if $\mathbf{K} = 4$ and we want to find the number of strings lexicographically smaller than $S' = "da"$. Converting S' from base-4 to base-10 will give $3 \times 4^1 + 0 \times 4^0 = 12$ which is the number of strings smaller than $"da"$ which are $["aa", "ab", "ac", "ad", "ba", "bb", "bc", "bd", "ca", "cb", "cc", "cd"]$.